



Term Project · Algorithm Design

Cherry Pickup

LeetCode 741 — solved with top-down DFS + memoization and bottom-up tabulation, with a comparison of both approaches

CSX-3009

Algorithm Design

543

Dr. KWANKAMOL NONGPONG

30 September 2026

LeetCode, problem 741 “Cherry Pickup” (Hard)

Team members

6511684 Min Pyae Hein

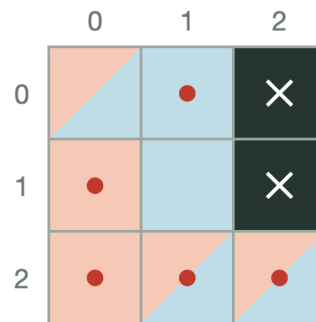
6611201 Aung Kaung Myat

Contents. 1 Problem description · 2 Key ideas · 3 Approach 1: top-down DFS + memoization · 4 Approach 2: bottom-up tabulation · 5 Complexity analysis · 6 Comparison of the two approaches · 7 Testing and online-judge submission · 8 Visualization tool · 9 Conclusion · 10 AI usage disclosure · References · Appendix: source code

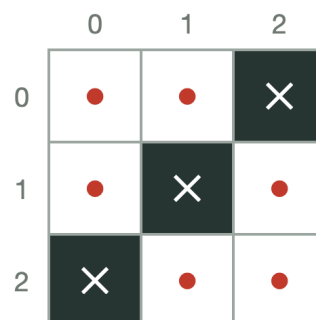
1. Problem description

We are given an $n \times n$ grid ($1 \leq n \leq 50$). Every cell holds one of three values: **0** — an empty cell that can be walked through, **1** — a cell with one cherry that can be picked up and walked through, and **-1** — a thorn that blocks the way. The top-left and bottom-right cells are never thorns.

A player starts at (0, 0), walks to (n-1, n-1) moving only **right or down** through non-thorn cells, and then walks back to (0, 0) moving only **left or up**. Whenever the player passes a cell with a cherry, the cherry is picked up and the cell becomes empty, so a cherry can be collected **at most once**. The task is to return the maximum number of cherries that can be collected. If there is no valid path between the two corners, the answer is 0.



Example 1: grid `[[0,1,-1],[1,0,-1],[1,1,1]]`, answer 5. ● = cherry, × = thorn. The shaded cells show one optimal pair of routes (A = orange, B = blue, shared = split).

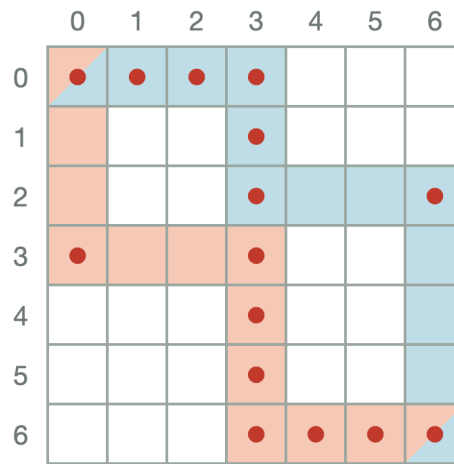


Example 2: grid `[[1,1,-1],[1,-1,1],[-1,1,1]]`. The thorns cut every path from the start to the finish, so the answer is 0.

2. Key ideas

2.1 Why a greedy approach fails

A natural idea is to take the best single path to the corner, remove its cherries, and then take the best path back. This greedy strategy is **not optimal**, because the first path does not consider what the return trip can still reach. On the 7×7 grid below, greedy collects **14** cherries while the optimum is **15** (both values computed by our benchmark script). The two trips must therefore be planned *together*.



Greedy counterexample. Planning both routes together (shaded) collects 15; greedy stops at 14.

2.2 Two walkers instead of a round trip

A path back from $(n-1, n-1)$ to $(0, 0)$ using left/up moves is the same set of cells as a path from $(0, 0)$ to $(n-1, n-1)$ using right/down moves. So the round trip is equivalent to **two walkers, A and B, that both start at $(0, 0)$ and walk to $(n-1, n-1)$ at the same time**, one step each per time unit. Both walks take exactly $2(n-1)$ steps.

2.3 State reduction: (step, row1, row2)

After t steps a walker at row r must be in column $c = t - r$, because every move increases $r + c$ by exactly one. So instead of four coordinates $(r1, c1, r2, c2)$ we only need **$(t, r1, r2)$** . All positions reachable at step t lie on the anti-diagonal $r + c = t$.

2.4 Counting a shared cell once

Because both walkers move in lock-step, they can only stand on the same cell at the same step. At step t , they are on the same cell exactly when $r1 = r2$. That cell's cherry is added once. A cell visited by A at one step and by B at a *different* step is impossible, since a cell (r, c) is only reachable at step $r + c$. This is what makes the “at most once” rule easy to enforce.

2.5 Transitions

Each walker moves either down (row + 1) or right (row unchanged), giving four combined moves: $(A\downarrow, B\downarrow)$, $(A\downarrow, B\rightarrow)$, $(A\rightarrow, B\downarrow)$, $(A\rightarrow, B\rightarrow)$. A move is valid only if both new cells are inside the grid and are not thorns. The cherries gained by a move are

$$\text{gain}(t, r1, r2) = \text{grid}[r1][t - r1] + (\text{grid}[r2][t - r2] \text{ if } r1 \neq r2 \text{ else } 0)$$

3. Approach 1 — DFS with memoization

3.1 Recurrence

Let **$F(t, r1, r2)$** be the maximum number of cherries that can *still* be collected after the walkers are at $(r1, t - r1)$ and $(r2, t - r2)$. This is the **best future** from the state.

```
F(t, r1, r2) both reach n-1,n-1 both 2 people.
F(t, r1, r2) = max over the 4 moves (d1, d2) that are valid of
               gain(t+1, r1+d1, r2+d2) + F(t+1, r1+d1, r2+d2)
F(t, r1, r2) = -∞ if no move is valid(not reaching to (n-1,n-1)(dead end)
               no cheery will return
```

```
answer = grid[0][0] + F(0, 0, 0), or 0 if F(0, 0, 0) = -∞
```

The recursion is a depth-first search: it follows one sequence of moves all the way to the finish, returns the value, and backtracks to try the next move. Many different move sequences reach the same state, so each F value is stored in a dictionary keyed by (t, r1, r2) and reused. That turns an exponential search into a polynomial one.

3.2 Why the memo must store the *future*, not the running total

Our first version (Cherry_DFS.py) passed the number of cherries collected *so far* down the recursion and returned that running total at the finish, while memoizing by (t, r1, r2) only. The stored value then depended on *which path* first reached the state. A later path with a different running total reused a number that was wrong for it. For example, on the grid $[[0,1,-1,-1],[0,1,0,1],[-1,1,0,1],[0,1,1,1]]$ it printed 6 instead of 8. Returning the best **future** (which depends only on the state) fixes this. The corrected version is the one presented in this report.

3.3 Algorithm

```
function DFS(t, r1, r2):
    if t = 2n - 2: return 0
    if (t, r1, r2) in memo: return memo[(t, r1, r2)]
    best ← -∞
    for (d1, d2) in {(1,1), (1,0), (0,1), (0,0)}:
        a ← r1 + d1; b ← r2 + d2; s ← t + 1
        if cells (a, s-a) and (b, s-b) are inside the grid and not thorns:
            best ← max(best, gain(s, a, b) + DFS(s, a, b))
    memo[(t, r1, r2)] ← best
    return best

answer ← max(0, grid[0][0] + DFS(0, 0, 0))
```

4. Approach 2 — Bottom-up tabulation

4.1 Recurrence

The bottom-up version fills the same state space forward in time. Let $G(t, r1, r2)$ be the maximum number of cherries collected *so far* when A is at (r1, t - r1) and B is at (r2, t - r2). A state can be entered from the state at step t - 1 in which each walker was either one row up (it moved down) or in the same row (it moved right):

```
G(0, 0, 0) = grid[0][0]
G(t, r1, r2) = gain(t, r1, r2) + max(
    G(t-1, r1, r2),      A→, B→
    G(t-1, r1-1, r2),    A↓, B→
    G(t-1, r1, r2-1),    A→, B↓
    G(t-1, r1-1, r2-1) ) A↓, B↓
G(t, r1, r2) = -∞ if either cell is a thorn or all four predecessors are -∞
answer = max(0, G(2n-2, n-1, n-1))
```

G is the “prefix” counterpart of F: for every state on an optimal route, $G + F$ equals the answer. Both describe the same DP. They only differ in the direction the table is filled.

4.2 Filling order and the rolling array

$G(t, \cdot, \cdot)$ depends only on $G(t - 1, \cdot, \cdot)$, so the table is filled one step (one anti-diagonal) at a time, and only two $n \times n$ layers are kept: **previous** and **current**. For each step only rows $\max(0, t - n + 1) \dots \min(n - 1, t)$ are valid, which skips impossible cells.

4.3 Worked example (Example 1)

The layers below are produced by the algorithm for Example 1. Hatched cells are rows that cannot occur at that step, and $-\infty$ marks unreachable pairs (a thorn or no valid predecessor). The final layer’s corner value is the answer, 5. The tables are symmetric because swapping A and B gives the same situation.

step 0				step 1				step 2				step 3				step 4			
r1\r2	0	1	2	r1\r2	0	1	2	r1\r2	0	1	2	r1\r2	0	1	2	r1\r2	0	1	2
0	0			0	1	2		0	$-\infty$	$-\infty$	$-\infty$	0				0			
1				1	2	1		1	$-\infty$	2	3	1		$-\infty$	$-\infty$	1			
2				2				2	$-\infty$	3	2	2		$-\infty$	4	2			5

For example, $G(2, 1, 2)$ puts A on (1, 1) and B on (2, 0). Its four candidates are $G(1, 1, 2) = -\infty$, $G(1, 0, 2) = -\infty$ (row 2 does not exist at step 1), $G(1, 1, 1) = 1$ and $G(1, 0, 1) = 2$. The best is 2, and the walkers add 0 + 1 cherries, so $G(2, 1, 2) = 3$.

5. Complexity analysis

5.1 Number of states

t ranges over $0 \dots 2n - 2$ ($2n - 1$ values) and $r1, r2$ over $0 \dots n - 1$, so there are at most $(2n - 1) \cdot n^2$ states. Only rows in the valid range for each step count. If $k_t = \min(t, 2n - 2 - t) + 1$ is the length of anti-diagonal t , the exact number of pairs is $\sum_t k_t^2 = (2n^3 + n)/3 \approx (2/3)n^3$. For $n = 50$ this is 83,350, which matches the 83,349 cells computed by the bottom-up solution on the open 50×50 grid.

5.2 Approach 2 — bottom-up

- **Time:** $\Theta(n^3)$. There are $2n - 1$ layers, at most n^2 cells per layer, and 4 predecessor lookups per cell, all $O(1)$.
- **Space:** $\Theta(n^2)$. Only two $n \times n$ layers are stored. The full 3-D table is never materialized.

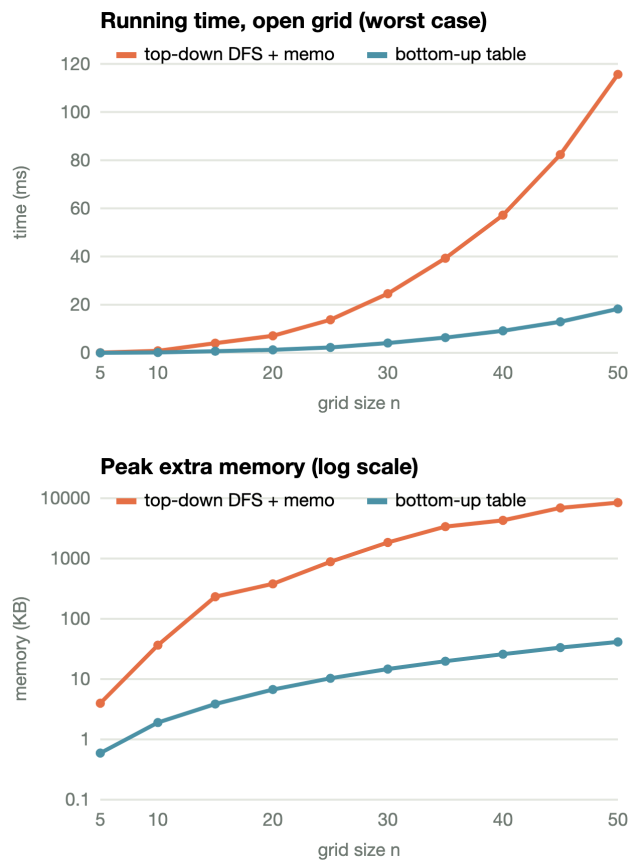
6. Comparison of the two approaches

6.1 Side by side

Aspect	Top-down DFS + memoization	Bottom-up tabulation
Value stored	F = best future from a state	G = best so far at a state
Direction	From the start, recursing towards the finish; values return backwards	Layer by layer from step 0 to step $2n - 2$
Order of evaluation	Automatic: the recursion discovers what it needs	Must be chosen by hand (by step)
States computed	Only states reachable from the start without thorns	Every non-thorn pair on each anti-diagonal, including unreachable ones
Time	$\Theta(n^3)$, with a larger constant: function calls, dict hashing of tuples	$\Theta(n^3)$, with a small constant: list indexing in loops
Space	$\Theta(n^3)$ memo + $\Theta(n)$ call stack	$\Theta(n^2)$ with two rolling layers
Risk	Recursion depth; memoizing a path-dependent value (see 3.2)	Boundary / index errors when reading row $- 1$
Recovering the route	Easy: follow the move that achieved each max	Needs parent pointers, or the full 3-D table
Ease of writing	Follows the recurrence directly; easiest to get right first	Needs more thought about order and bounds

6.2 Experimental results

Both solutions were run on the same machine (Python 3.14.6), timed with `time.perf_counter` (median of 3–5 runs) and measured with `tracemalloc` for peak extra memory. “Open” grids contain only cherries (no thorns), which is the worst case because every state is reachable. “Random” grids have about 15% thorns and 50% cherries. The script is `benchmark.py`.



n	answer	Top-down DFS + memo			Bottom-up tabulation		
		dfs calls	time (ms)	peak mem (KB)	cells	time (ms)	peak mem (KB)
5	16	241	0.09	4.0	84	0.02	0.6
10	36	2,281	0.83	36.5	669	0.17	1.9
15	56	8,121	3.96	231.8	2,254	0.67	3.9
20	76	19,761	7.11	380.4	5,339	1.23	6.7
25	96	39,201	13.70	882.1	10,424	2.27	10.3
30	116	68,441	24.53	1,844.6	18,009	4.07	14.6
35	136	109,481	39.31	3,395.7	28,594	6.29	19.9
40	156	164,321	57.17	4,283.7	42,679	9.13	25.9
45	176	234,961	82.33	6,918.6	60,764	12.90	33.2
50	196	323,401	115.65	8,423.7	83,349	18.23	41.2

Table: open grids (worst case).

n	answer	Top-down DFS + memo			Bottom-up tabulation		
		dfs calls	time (ms)	peak mem (KB)	cells	time (ms)	peak mem (KB)
5	8	120	0.06	4.7	60	0.02	0.6
10	29	1,149	0.49	55.3	497	0.23	1.9
15	40	2,531	1.01	55.3	1,765	0.45	3.9
20	62	7,000	2.94	424.9	3,540	0.96	6.7
25	80	5,278	2.13	218.7	7,446	1.63	10.3
30	90	14,370	6.39	874.0	12,392	3.03	14.6
35	115	39,034	16.37	1,553.2	19,854	5.06	19.9
40	128	38,339	15.51	1,732.0	30,248	6.94	25.9
45	135	75,498	31.32	3,687.4	43,369	10.45	33.2
50	156	105,540	41.45	4,305.7	61,498	14.04	41.2

Table: random grids ($\approx 15\%$ thorns).

6.3 Discussion

- **Both grow as n^3 .** From $n = 25$ to $n = 50$ (n doubles), the open-grid time grows by $8.4\times$ (top-down) and $8.0\times$ (bottom-up), close to the $2^3 = 8\times$ predicted by $\Theta(n^3)$.
- **Bottom-up is faster in practice**, about $6.3\times$ at $n = 50$ on the open grid. Both do $\Theta(n^3)$ work, but a Python function call plus a tuple-keyed dictionary lookup is much more expensive than indexing a list inside a loop.
- **Bottom-up uses far less memory:** 41 KB against 8,424 KB at $n = 50$, about $204\times$ less. This matches $\Theta(n^2)$ against $\Theta(n^3)$. The memo dictionary also stores a tuple key per entry.
- **Top-down benefits from thorns.** On random grids, the top-down search only reaches states connected to the start, so its gap to bottom-up shrinks. At $n = 50$ the ratio drops from $6.3\times$ to $3.0\times$. Bottom-up still evaluates every non-thorn pair on each diagonal.
- **Both are fast enough.** With $n \leq 50$ the worst case is about 83,000 states, so both solutions finish in well under a second, far inside LeetCode's time limit.

7. Testing and online-judge submission

- **Brute-force cross-check.** Both solutions were compared with an exhaustive search over every pair of paths on 3,000 random grids ($n = 1 \dots 5$, about 20% thorns). Result: 6,000 checks and **0 mismatches**.
- **LeetCode examples.** Example 1 $\rightarrow 5 / 5$ (expected 5); Example 2 $\rightarrow 0 / 0$ (expected 0).
- **Agreement.** On every benchmark grid (n up to 50) the two solutions returned the same answer.

LeetCode submission	Status	Runtime	Memory
Approach 1 — top-down DFS + memoization	[Accepted]	[941 ms]	[74.67 MB]
Approach 2 — bottom-up tabulation	[Accepted]	[254 ms]	[19.50 MB]

Accepted 60 / 60 testcases passed

 norman7781 submitted at Sep 30, 2026 20:41

 Analysis

 Solution



Last Call. Leet's On.

Grab our Back to School student deal before it's gone!

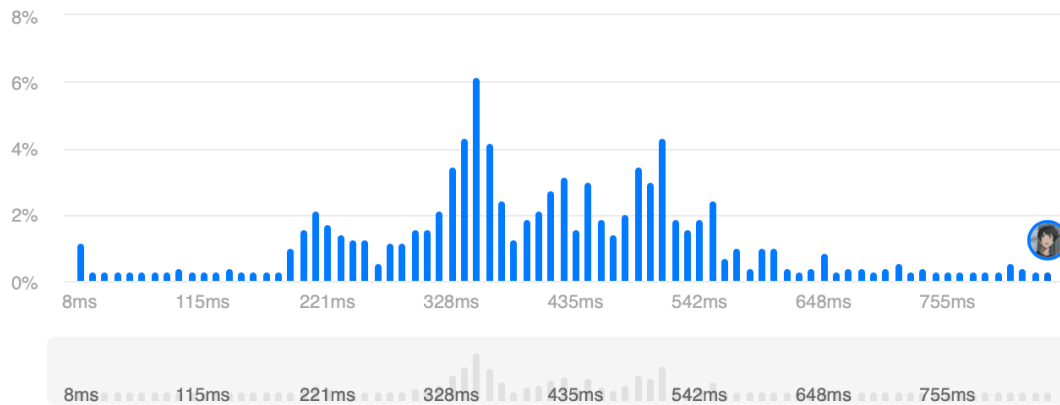


🕒 Runtime

941 ms | Beats 5.03%

⚙️ Memory

74.67 MB | Beats 26.04%



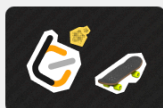
Code | Python3

Accepted 60 / 60 testcases passed

 norman7781 submitted at Sep 30, 2026 21:06

 Analysis

 Solution



Last Call. Leet's On.

Grab our Back to School student deal before it's gone!

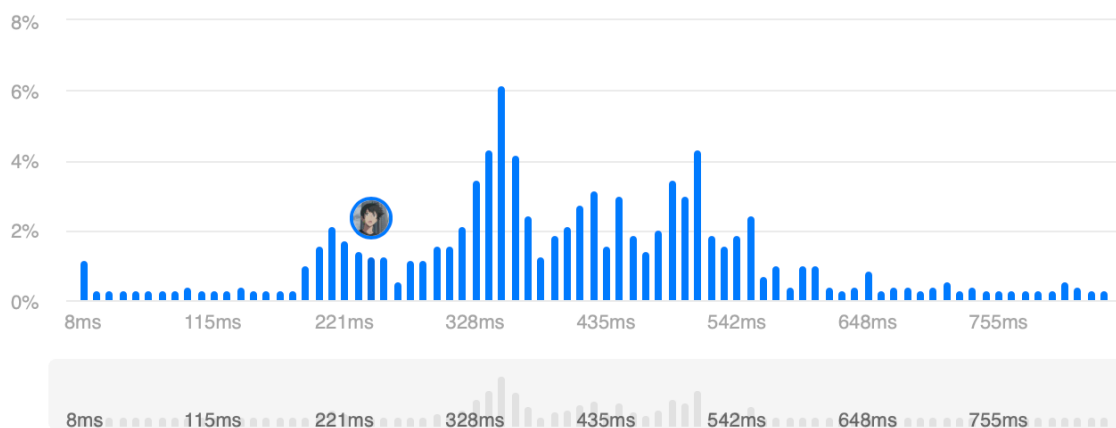


🕒 Runtime

245 ms | Beats 87.05% 🌱

⚙️ Memory

19.50 MB | Beats 87.63% 🌱



8. Visualization tool

To explain both algorithms in the presentation, we built an interactive web application (Next.js / React).

- **DFS + memoization page.** The recursion tree grows one call at a time. It highlights the current call, the calls waiting on the stack, memo hits, base cases and dead ends, and shows each return as “max(gain + future, ...)”. At the end it highlights the best route in the tree and on the grid.
- **Bottom-up page.** Shows the previous → current layers, the four candidate cells for each entry, the formula being evaluated, and the matching line of Python code. It has a *predict mode* in which the viewer fills in each cell before it is revealed.

9. Conclusion

Cherry Pickup looks like two separate path problems, but the trips interact through shared cherries, so greedy fails. Modelling the round trip as two walkers moving together, and using the fact that a walker's column is determined by the step and its row, reduces the state to (step, row1, row2) and gives an $O(n^3)$ dynamic program. We implemented it both ways. The top-down version maps directly onto the recurrence and only explores reachable states. The bottom-up version has the same $\Theta(n^3)$ time but a smaller constant factor, and it needs only $\Theta(n^2)$ memory thanks to the rolling layers. In our measurements it was about 6× faster and used about 204× less memory at $n = 50$. We also learned that the value a memo stores must depend only on the state. Otherwise memoization silently returns wrong answers.

10. AI usage disclosure

Tool used. Claude Code, Anthropic's AI coding assistant (model: Claude Opus 5.5), used in the terminal inside our project folder.

What we did ourselves. We analysed the problem and solved it ourselves: we worked out the two-walker idea and the (step, row1, row2) state, and we chose the two algorithms to compare, top-down DFS with memoization and bottom-up tabulation. We wrote the original solutions for both approaches (Cherry_DFS.py and the bottom-up code).

How the AI assisted us. Mainly with building the visualization website; smaller help is listed after it.

- Built the interactive visualization website (Next.js / React): the DFS + memoization page with the step-by-step recursion tree and route highlighting, and the bottom-up table page. It also explained how to run the project.
- Found a memoization bug in our first top-down code (Cherry_DFS.py) and wrote the corrected version (cherry_pickup_dfs_memo.py).
- Put our bottom-up solution into the LeetCode Solution class format with a local test driver (cherry_pickup_bottom_up.py). The algorithm itself is ours and unchanged.
- Wrote the benchmark / testing script (benchmark.py).
- Helped draft and format this report, which we then reviewed and edited. **Responsibility.** We have reviewed, tested and understood all code and text in this submission, including every part produced with AI assistance, and we take full responsibility for the final output.

References

1. LeetCode, “741. Cherry Pickup”. <https://leetcode.com/problems/cherry-pickup/>
2. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 4th ed., MIT Press, 2022 — Chapter 14, Dynamic Programming.

3. J. Kleinberg, É. Tardos. *Algorithm Design*, Pearson, 2006 — Chapter 6, Dynamic Programming.

Appendix A — cherry_pickup_dfs_memo.py

```
"""
```

Cherry Pickup (LeetCode 741) – Approach 1: top-down DFS with memoization

```
class State:
```

```
    def __init__(self, row1, row2, step):
```

```
        self.row1 = row1
```

```
        self.row2 = row2
```

```
        self.step = step
```

```
class Solution:
```

```
    def cherryPickup(self, grid: list[list[int]]) -> int:
```

```
        n = len(grid)
```

```
        memo = {}
```

```
    def valid(row1, row2, step):
```

```
        col1 = step - row1
```

```
        col2 = step - row2
```

```
        if row1 >= n or col1 >= n or row2 >= n or col2 >= n:
```

```
            return False
```

```
        if grid[row1][col1] == -1 or grid[row2][col2] == -1:
```

```
            return False
```

```
        return True
```

```
    def successor(s):
```

```
        succ = []
```

```
        moves = [(1, 1), (1, 0), (0, 1), (0, 0)] # (d_row1, d_row2)
```

```
        for dr1, dr2 in moves:
```

```
            nr1 = s.row1 + dr1
```

```
            nr2 = s.row2 + dr2
```

```
            nstep = s.step + 1
```

```
            if valid(nr1, nr2, nstep):
```

```
                ncol1 = nstep - nr1
```

```
                ncol2 = nstep - nr2
```

```
                gain = grid[nr1][ncol1]
```

```
                if (nr1, ncol1) != (nr2, ncol2):
```

```
                    gain += grid[nr2][ncol2]
```

```
                succ.append((State(nr1, nr2, nstep), gain))
```

```
        return succ
```

```
    def dfs(s):
```

```
if s.step == 2 * (n - 1):
    return 0

key = (s.step, s.row1, s.row2)
if key in memo:
    return memo[key]

best = float('-inf')
for u, gain in successor(s):
    best = max(best, gain + dfs(u))

memo[key] = best
return best

result = grid[0][0] + dfs(State(0, 0, 0))
return max(result, 0)
```

Appendix B — cherry_pickup_bottom_up.py

```
"""
```

Cherry Pickup (LeetCode 741) – Approach 2: bottom-up tabulation

Idea: same two-walker state (step, row1, row2) as the top-down version, but the table is filled forward one step (anti-diagonal) at a time, with no recursion. `previous[row1][row2]` holds the most cherries collected so far when walker A is in row1 and walker B is in row2. Only the previous layer is needed to build the current one, so two $n \times n$ tables are enough.

Time $O(n^3)$: $(2n-1)$ layers, $n \times n$ cells each, 4 predecessors per cell.

Space $O(n^2)$: two $n \times n$ layers.

```
"""
```

```
import time
from typing import List

class Solution:
    def cherryPickup(self, grid: List[List[int]]) -> int:
        n = len(grid)
        negative_infinity = float("-inf")
        self.state_count = 0

        # At step 0, both walkers are at (0, 0)
        previous = [[negative_infinity] * n for _ in range(n)]
        previous[0][0] = grid[0][0]

        # Both walkers need 2(n - 1) steps to reach the destination
        for step in range(1, 2 * n - 1):
            current = [[negative_infinity] * n for _ in range(n)]

            minimum_row = max(0, step - (n - 1))
            maximum_row = min(n - 1, step)

            for row1 in range(minimum_row, maximum_row + 1):
                column1 = step - row1

                if grid[row1][column1] == -1:
                    continue

                for row2 in range(minimum_row, maximum_row + 1):
                    column2 = step - row2

                    if grid[row2][column2] == -1:
                        continue

                    self.state_count += 1
                    best_previous = previous[row1][row2]

                    if row1 > 0:
                        best_previous = max(best_previous, previous[row1 - 1][row2])

                    if row2 > 0:
                        best_previous = max(best_previous, previous[row1][row2 - 1])

                    if row1 > 0 and row2 > 0:
                        best_previous = max(
                            best_previous, previous[row1 - 1][row2 - 1]
                        )

                    if best_previous == negative_infinity:
                        continue

                    cherries = grid[row1][column1]

                    # Count the second cell only when positions differ
                    if row1 != row2:
```

```

        cherries += grid[row2][column2]

        current[row1][row2] = best_previous + cherries

        previous = current

    return max(0, previous[n - 1][n - 1])

if __name__ == "__main__":
    # Input: n, then n rows of n integers.
    n = int(input())
    grid = [list(map(int, input().split())) for _ in range(n)]

    solver = Solution()
    t0 = time.perf_counter()
    answer = solver.cherryPickup(grid)
    t1 = time.perf_counter()

    print(answer)
    print("cells computed =", solver.state_count)
    print("time taken      = %.6f sec" % (t1 - t0))

```